



ZAAWANSOWANE PROGRAMOWANIE W JĘZYKU C++

RAPORT Z WYKONANIA PROJEKTU

PLANE MANAGER

Hubert Lech — 177827 — L09

2026-05-16

Wstęp

Plane Manager to desktopowa aplikacja do zarządzania niewielką flotą lotniczą, zaprojektowana tak, aby w prosty i czytelny sposób zarządzać informacjami o samolotach oraz ich wykorzystaniu w zaplanowanych lotach. Z perspektywy użytkownika aplikacja działa jak mały panel administracyjny: można dodać samolot, uzupełnić jego parametry techniczne (np. długość, liczba silników, pojemność pasażerska, prędkość i pułap), zmienić status (np. dostępny lub w serwisie), a następnie zaplanować lot pomiędzy wybranymi lotniskami. Podczas rezerwacji i edycji lotów aplikacja pilnuje porządku w terminarzu — sprawdza, czy daty są poprawne oraz czy nie dochodzi do nakładania się rezerwacji dla tego samego samolotu; dodatkowo uwzględnia sytuację, w której samolot jest wyłączony z użytku (np. znajduje się w serwisie). Poza samym planowaniem lotów dostępny jest też widok podsumowań: statystyki floty (ile maszyn jest dostępnych, w locie lub w serwisie), a także informacje takie jak najdłuższy i najkrótszy samolot czy największa i najmniejsza liczba pasażerów. Dzięki temu jednym spojrzeniem można ocenić, w jakiej kondycji jest flota i jakie maszyny wyróżniają się parametrami. Projekt został zrealizowany w oparciu o framework Qt. Logika aplikacji i komunikacja z bazą danych zostały napisane w C++, natomiast interfejs użytkownika przygotowano w QML, co pozwoliło uzyskać responsywny i miły dla oka układ widoków. Dane przechowywane są w relacyjnej bazie PostgreSQL uruchomionej w środowisku Supabase, a dostęp do nich realizowany jest przez moduł QSql.

Baza danych

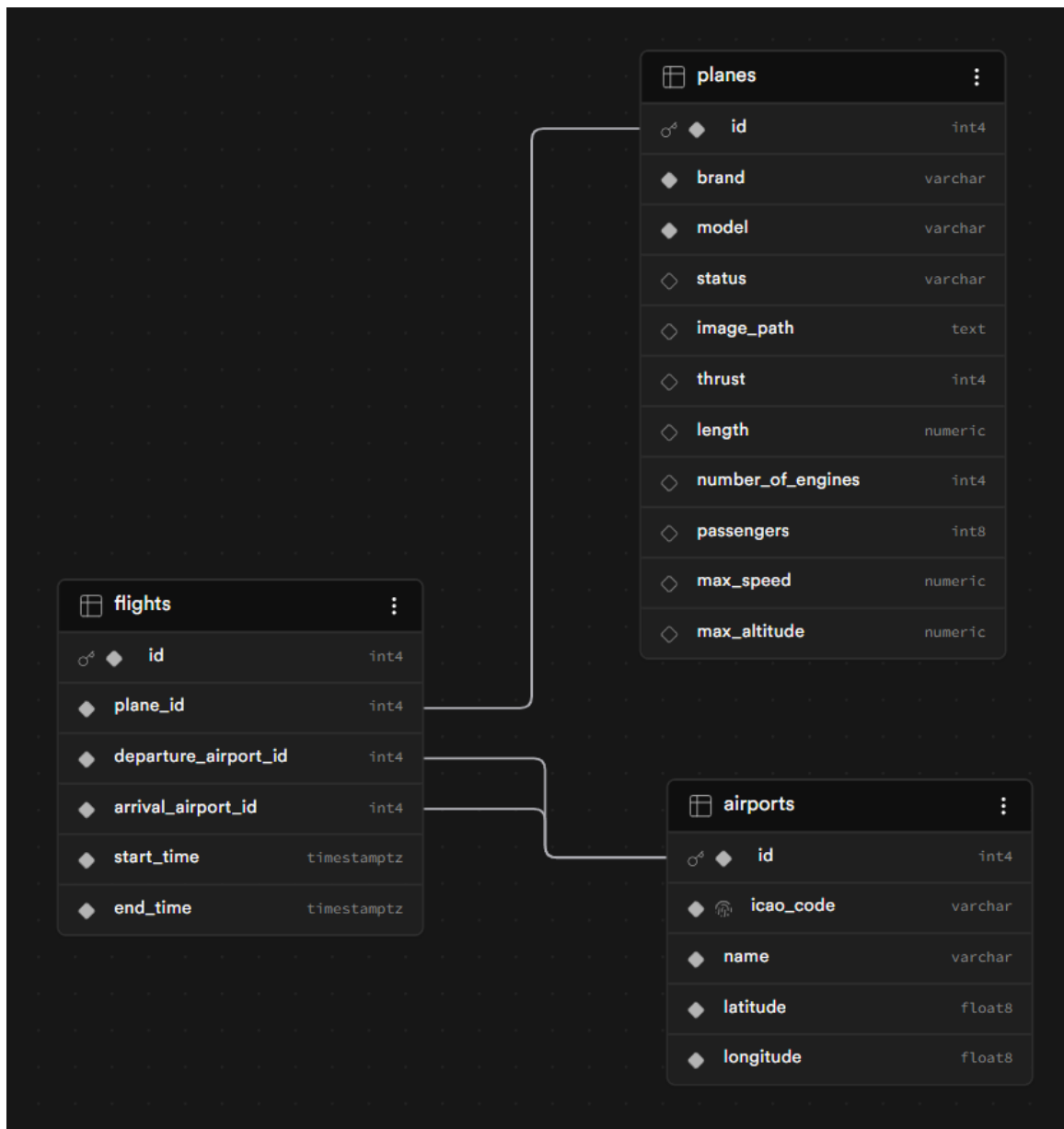
Po uruchomieniu aplikacji nawiązywane jest połączenie z bazą danych PostgreSQL działającą w Supabase. Za ten etap odpowiada klasa `DatabaseManager`, która konfiguruje połączenie w QtSql z użyciem sterownika QPSQL. Parametry dostępu nie są wpisane na stałe w kodzie — aplikacja pobiera je ze zmiennych środowiskowych (m.in. `DB_HOST`, `DB_NAME`, `DB_USER`, `DB_PASS`, `DB_PORT`), co ułatwia przenoszenie projektu pomiędzy komputerami i ogranicza ryzyko przypadkowego ujawnienia danych dostępowych w repozytorium. Dodatkowo ustawione zostały opcje połączenia takie jak `keepalives` i limit czasu na nawiązanie połączenia, aby zmniejszyć prawdopodobieństwo rozłączania sesji na warstwie poolera.

Warstwa danych aplikacji została oparta o relacyjną bazę PostgreSQL uruchomioną w środowisku Supabase. Taki wybór pozwala przechowywać dane w sposób uporządkowany (tabele i relacje), a jednocześnie ułatwia dalszą rozbudowę projektu o kolejne encje. Po stronie aplikacji połączenie realizowane jest przez QtSql z wykorzystaniem sterownika QPSQL, a zapytania wykonywane są w większości jako `prepared statements` (`QSqlQuery::prepare` oraz `bindValue`), co poprawia czytelność kodu i ogranicza ryzyko błędów związanych z wklejaniem wartości do SQL.

W projekcie baza danych składa się z trzech głównych tabel, które odzwierciedlają podstawowe elementy domeny:

- **planes** — przechowuje informacje o samolotach (marka, model, status oraz parametry techniczne takie jak długość, liczba silników, pojemność, prędkość maksymalna i pułap). Rekordy identyfikowane są kluczem głównym `id`.
- **airports** — zawiera lotniska wraz z kodem ICAO, nazwą oraz współrzędnymi geograficznymi (szerokość i długość). Rekordy identyfikowane są kluczem `id`; kod `icao_code` jest wykorzystywany do czytelnej prezentacji w UI.
- **flights** — reprezentuje zaplanowane loty: wskazuje samolot (`plane_id`) oraz lotniska wylotu i przylotu (`departure_airport_id`, `arrival_airport_id`), a także przedział czasowy lotu (`start_time`, `end_time`).

Relacje pomiędzy tabelami są naturalne dla problemu: jeden samolot może wystąpić w wielu lotach (relacja 1–N), a każdy lot wskazuje dwa lotniska (wylotu i przylotu). Schemat relacji przedstawiono na diagramie 1. W czasie implementacji ważne było również poprawne przetwarzanie czasu — wartości czasowe zapisywane są w bazie w UTC (aplikacja dokonuje konwersji na UTC przy zapisie oraz na czas lokalny przy prezentacji), a przy dodawaniu/edycji lotu wykonywana jest walidacja terminów, w tym wykrywanie nakładania się rezerwacji dla tego samego samolotu.



Rysunek 1: Diagram relacji bazy danych (ERD)

Poniżej przedstawiono przykładowe fragmenty kodu odpowiedzialne za operacje CRUD (Create, Read, Update, Delete) na głównych encjach aplikacji: samolotach, lotniskach oraz lotach.

Samoloty (planes). Listing 1 pokazuje kompletny zestaw operacji na tabeli **planes**. Dodawanie i aktualizacja realizowane są poprzez zapytania parametryzowane (**prepare + bindValue**), co upraszcza przekazywanie wielu pól technicznych samolotu i zachowuje porządek w kodzie. Odczyt (**getAllPlanes**) pobiera listę samolotów w postaci **QVariantList** z mapami, gotową do bezpośredniego użycia w warstwie QML. Dodatkowo w zapytaniu odczytu zastosowano logikę w SQL (**CASE + EXISTS**), aby status samolotu w UI uwzględniał to, czy maszyna aktualnie bierze udział w locie.

```

1  bool PlaneManager::addPlane(const QString &brand, const QString
    &model, const QString &status,
2                                int thrust, double length, int
                                numberOfEngines,
  
```

```

3         int passengers, double maxSpeed, double
           maxAltitude) {
4
5     QSqlQuery query;
6     query.prepare("INSERT INTO planes (brand, model, status, thrust,
           length, number_of_engines, passengers, max_speed, max_altitude,
           image_path) "
7         "VALUES (:brand, :model, :status, :thrust, :length,
           :numberOfEngines, :passengers, :maxSpeed,
           :maxAltitude, '')");
8     query.bindValue(":brand", brand);
9     query.bindValue(":model", model);
10    query.bindValue(":status", status);
11    query.bindValue(":thrust", thrust);
12    query.bindValue(":length", length);
13    query.bindValue(":numberOfEngines", numberOfEngines);
14    query.bindValue(":passengers", passengers);
15    query.bindValue(":maxSpeed", maxSpeed);
16    query.bindValue(":maxAltitude", maxAltitude);
17
18    if (!query.exec()) {
19        qDebug() << "Blad SQL przy dodawaniu samolotu:" <<
           query.lastError().text();
20        return false;
21    }
22
23    qDebug() << "Pomyślnie dodano samolot:" << brand << model;
24    return true;
25 }
26
27 QVariantList PlaneManager::getAllPlanes() {
28     QVariantList list;
29     QSqlQuery query(
30         "SELECT p.id, p.brand, p.model, p.thrust, p.length,
31             p.number_of_engines, p.passengers, p.max_speed,
32             p.max_altitude, "
33         "COALESCE(p.image_path, '') as image_path, "
34         "CASE "
35         "    WHEN EXISTS ("
36         "        SELECT 1 FROM flights f "
37         "        WHERE f.plane_id = p.id "
38         "        AND f.start_time <= CURRENT_TIMESTAMP "
39         "        AND f.end_time >= CURRENT_TIMESTAMP "
40         "    ) THEN 'W locie' "
41         "    WHEN LOWER(COALESCE(p.status, '')) = 'w locie' THEN
           'Dostepny' "
42         "    ELSE p.status "
43         "END AS status "
44         "FROM planes p "
45         "ORDER BY p.id DESC"
46     );
47
48     while (query.next()) {

```

```

46     QVariantMap map;
47     map["id"] = query.value("id");
48     map["brand"] = query.value("brand");
49     map["model"] = query.value("model");
50     map["status"] = query.value("status");
51     map["thrust"] = query.value("thrust");
52     map["length"] = query.value("length");
53     map["numberOfEngines"] = query.value("number_of_engines");
54     map["passengers"] = query.value("passengers");
55     map["maxSpeed"] = query.value("max_speed");
56     map["maxAltitude"] = query.value("max_altitude");
57     map["imagePath"] = query.value("image_path");
58     list.append(map);
59 }
60 return list;
61 }
62
63 bool PlaneManager::updatePlane(int id, const QString &brand, const
    QString &model, const QString &status,
64                                int thrust, double length, int
    numberOfEngines,
65                                int passengers, double maxSpeed,
    double maxAltitude, const QString
    &imagePath) {
66     QSqlQuery query;
67     query.prepare("UPDATE planes SET brand = :brand, model = :model,
    status = :status, thrust = :thrust, length = :length, "
68                  "number_of_engines = :numberOfEngines, passengers =
    :passengers, max_speed = :maxSpeed, max_altitude
    = :maxAltitude, "
69                  "image_path = :imagePath WHERE id = :id");
70     query.bindValue(":brand", brand);
71     query.bindValue(":model", model);
72     query.bindValue(":status", status);
73     query.bindValue(":thrust", thrust);
74     query.bindValue(":length", length);
75     query.bindValue(":numberOfEngines", numberOfEngines);
76     query.bindValue(":passengers", passengers);
77     query.bindValue(":maxSpeed", maxSpeed);
78     query.bindValue(":maxAltitude", maxAltitude);
79     query.bindValue(":imagePath", imagePath.isEmpty() ? QVariant() :
    imagePath);
80     query.bindValue(":id", id);
81
82     if (!query.exec()) {
83         qDebug() << "Bład SQL przy aktualizacji samolotu:" <<
    query.lastError().text();
84         return false;
85     }
86     qDebug() << "Pomyślnie zaktualizowano samolot ID:" << id;
87     return true;
88 }

```

```

89
90 bool PlaneManager::deletePlane(int id) {
91     QSqlQuery query;
92     query.prepare("DELETE FROM planes WHERE id = :id");
93     query.bindValue(":id", id);
94
95     if (!query.exec()) {
96         qDebug() << "Bład SQL przy usuwaniu samolotu:" <<
97             query.lastError().text();
98         return false;
99     }
100     qDebug() << "Pomyślnie usunięto samolot ID:" << id;
101     return true;
102 }

```

Listing 1: Operacje CRUD dla samolotów (tabela planes)

Lotniska (airports). Listing 2 prezentuje podstawowe operacje CRUD dla lotnisk. Model danych jest tu prosty (kod ICAO, nazwa oraz współrzędne), dlatego zapytania są krótkie i czytelne: dodanie rekordu, pobranie listy posortowanej po kodzie ICAO, edycja oraz usunięcie po identyfikatorze id. Tak przygotowana warstwa pozwala UI szybko zasilać listy wyboru lotnisk oraz przechowywać dane potrzebne np. do prezentacji na mapie.

```

1 bool AirportManager::addAirport(const QString &icao, const QString
  &name, double lat, double lon) {
2     QSqlQuery query;
3     query.prepare("INSERT INTO airports (icao_code, name, latitude,
4         longitude) "
5         "VALUES (:icao, :name, :lat, :lon)");
6     query.bindValue(":icao", icao);
7     query.bindValue(":name", name);
8     query.bindValue(":lat", lat);
9     query.bindValue(":lon", lon);
10
11     if (!query.exec()) {
12         qDebug() << "Bład SQL przy dodawaniu lotniska:" <<
13             query.lastError().text();
14         return false;
15     }
16     qDebug() << "Pomyślnie dodano lotnisko:" << icao;
17     return true;
18 }
19
20 QVariantList AirportManager::getAllAirports() {
21     QVariantList list;
22     QSqlQuery query("SELECT id, icao_code, name, latitude, longitude
23         FROM airports ORDER BY icao_code ASC");
24
25     while (query.next()) {
26         QVariantMap map;
27         map["id"] = query.value("id");
28         map["icaoCode"] = query.value("icao_code");
29         map["name"] = query.value("name");

```

```

27     map["latitude"] = query.value("latitude");
28     map["longitude"] = query.value("longitude");
29     list.append(map);
30 }
31 return list;
32 }
33
34 bool AirportManager::updateAirport(int id, const QString &icao, const
    QString &name, double lat, double lon) {
35     QSqlQuery query;
36     query.prepare("UPDATE airports SET icao_code = :icao, name =
        :name, "
37                   "latitude = :lat, longitude = :lon WHERE id = :id");
38     query.bindValue(":icao", icao);
39     query.bindValue(":name", name);
40     query.bindValue(":lat", lat);
41     query.bindValue(":lon", lon);
42     query.bindValue(":id", id);
43
44     if (!query.exec()) {
45         qDebug() << "Bład SQL przy edycji lotniska:" <<
            query.lastError().text();
46         return false;
47     }
48     return true;
49 }
50
51 bool AirportManager::deleteAirport(int id) {
52     QSqlQuery query;
53     query.prepare("DELETE FROM airports WHERE id = :id");
54     query.bindValue(":id", id);
55
56     if (!query.exec()) {
57         qDebug() << "Bład SQL przy usuwaniu lotniska:" <<
            query.lastError().text();
58         return false;
59     }
60     return true;
61 }

```

Listing 2: Operacje CRUD dla lotnisk (tabela airports)

Loty (flights). Listing 3 zawiera operacje CRUD dla lotów, ale w tym przypadku sama „goła” manipulacja danymi nie wystarcza — najważniejsza jest walidacja reguł biznesowych. Przed dodaniem lub aktualizacją lotu aplikacja sprawdza m.in. czy wybrany samolot nie ma statusu serwisowego oraz czy jego terminarz nie koliduje z innymi lotami (nakładanie się przedziałów czasu). Odczyt listy lotów korzysta z JOIN, aby poza identyfikatorami zwrócić też czytelne dane (np. marka/model samolotu, kody ICAO i współrzędne lotnisk), a dodatkowo wylicza status lotu (zaplanowany/w trakcie/zakończony) na podstawie czasu.

```

1 #include "FlightManager.h"
2
3 FlightManager::FlightManager(QObject *parent) : QObject(parent) {}

```



```

4
5 namespace {
6 QDateTime toDbUtc(const QDateTime &dateTime) {
7     if (!dateTime.isValid()) {
8         return dateTime;
9     }
10    return dateTime.toUTC();
11 }
12
13 QDateTime toLocalForDisplay(const QVariant &value) {
14     QDateTime dateTime = value.toDateTime();
15     if (!dateTime.isValid()) {
16         return dateTime;
17     }
18
19     if (dateTime.timeSpec() == Qt::UTC || dateTime.timeSpec() ==
20         Qt::OffsetFromUTC) {
21         return dateTime.toLocalTime();
22     }
23
24     return dateTime;
25 }
26
27 QDateTime toUtcForCompare(const QVariant &value) {
28     QDateTime dateTime = value.toDateTime();
29     if (!dateTime.isValid()) {
30         return dateTime;
31     }
32     if (dateTime.timeSpec() == Qt::UTC || dateTime.timeSpec() ==
33         Qt::OffsetFromUTC) {
34         return dateTime.toUTC();
35     }
36
37     return dateTime.toUTC();
38 }
39
40 bool isPlaneInService(int planeId) {
41     QSqlQuery query;
42     query.prepare("SELECT LOWER(COALESCE(status, '')) AS status FROM
43         planes WHERE id = :planeId");
44     query.bindValue(":planeId", planeId);
45
46     if (!query.exec()) {
47         qDebug() << "Bład SQL przy sprawdzaniu statusu samolotu:" <<
48             query.lastError().text();
49         return true;
50     }
51
52     if (!query.next()) {
53         qDebug() << "Nie znaleziono samolotu o ID:" << planeId;
54         return true;
55     }
56 }

```

```

52     return query.value("status").toString() == "w serwisie";
53 }
54
55 bool hasPlaneScheduleConflict(int planeId, const QDateTime &startUtc,
56     const QDateTime &endUtc, int excludedFlightId = -1) {
57     if (!startUtc.isValid() || !endUtc.isValid() || endUtc <=
58         startUtc) {
59         return true;
60     }
61     QSqlQuery query;
62     if (excludedFlightId >= 0) {
63         query.prepare("SELECT 1 FROM flights "
64             "WHERE plane_id = :planeId AND id <> :excludedId "
65             "AND start_time < :endUtc AND end_time > "
66             ":startUtc "
67             "LIMIT 1");
68         query.bindValue(":excludedId", excludedFlightId);
69     } else {
70         query.prepare("SELECT 1 FROM flights "
71             "WHERE plane_id = :planeId "
72             "AND start_time < :endUtc AND end_time > "
73             ":startUtc "
74             "LIMIT 1");
75     }
76     query.bindValue(":planeId", planeId);
77     query.bindValue(":startUtc", startUtc);
78     query.bindValue(":endUtc", endUtc);
79
80     if (!query.exec()) {
81         qDebug() << "Bład SQL przy sprawdzaniu kolizji lotu:" <<
82             query.lastError().text();
83         return true;
84     }
85
86     return query.next();
87 }
88
89 bool FlightManager::addFlight(int planeId, int depAirportId, int
90     arrAirportId,
91     const QDateTime &startTime, const
92     QDateTime &endTime) {
93     const QDateTime startUtc = toDbUtc(startTime);
94     const QDateTime endUtc = toDbUtc(endTime);
95
96     if (isPlaneInService(planeId)) {
97         qDebug() << "Nie mozna zarezerwować lotu: samolot jest w
98             serwisie.";
99         return false;

```

```

95     }
96
97     if (hasPlaneScheduleConflict(planeId, startUtc, endUtc)) {
98         qDebug() << "Nie mozna zarezerwować lotu: samolot ma juz
99             zajety termin.";
100         return false;
101     }
102
103     QSqlQuery query;
104     query.prepare("INSERT INTO flights (plane_id,
105         departure_airport_id, arrival_airport_id, start_time, end_time)
106         "
107             "VALUES (:pId, :depId, :arrId, :start, :end)");
108     query.bindValue(":pId", planeId);
109     query.bindValue(":depId", depAirportId);
110     query.bindValue(":arrId", arrAirportId);
111     query.bindValue(":start", startUtc);
112     query.bindValue(":end", endUtc);
113
114     if (!query.exec()) {
115         qDebug() << "Błąd SQL przy dodawaniu lotu:" <<
116             query.lastError().text();
117         return false;
118     }
119     qDebug() << "Pomyślnie zarezerwowano lot.";
120     return true;
121 }
122
123 QVariantList FlightManager::getAllFlights() {
124     QVariantList list;
125     // Pobieramy ID, ale też czytelne nazwy dzięki JOIN
126     QString sql = "SELECT f.id, p.brand, p.model, "
127         "dep.icao_code as dep_icao, arr.icao_code as
128         arr_icao, "
129         "dep.latitude as dep_latitude, dep.longitude as
130         dep_longitude, "
131         "arr.latitude as arr_latitude, arr.longitude as
132         arr_longitude, "
133         "f.start_time, f.end_time, "
134         "f.plane_id, f.departure_airport_id,
135         f.arrival_airport_id "
136         "FROM flights f "
137         "JOIN planes p ON f.plane_id = p.id "
138         "JOIN airports dep ON f.departure_airport_id =
139         dep.id "
140         "JOIN airports arr ON f.arrival_airport_id = arr.id "
141         "ORDER BY f.start_time DESC";
142
143     QSqlQuery query(sql);
144     const QDateTime nowUtc = QDateTime::currentDateTimeUtc();
145     while (query.next()) {
146         const QDateTime startUtc =

```

```

138         toUtcForCompare(query.value("start_time"));
139         const QDateTime endUtc =
140             toUtcForCompare(query.value("end_time"));
141
142         QString statusText = "Zaplanowany";
143         QString statusColor = "#2E7D32";
144
145         if (startUtc.isValid() && endUtc.isValid()) {
146             if (nowUtc < startUtc) {
147                 statusText = "Zaplanowany";
148                 statusColor = "#2E7D32";
149             } else if (nowUtc <= endUtc) {
150                 statusText = "W trakcie";
151                 statusColor = "#F9A825";
152             } else {
153                 statusText = "Zakończony";
154                 statusColor = "#C62828";
155             }
156         }
157
158         QVariantMap map;
159         map["id"] = query.value("id");
160         map["planeName"] = query.value("brand").toString() + " " +
161             query.value("model").toString();
162         map["depIcao"] = query.value("dep_icao");
163         map["arrIcao"] = query.value("arr_icao");
164         map["depLatitude"] = query.value("dep_latitude");
165         map["depLongitude"] = query.value("dep_longitude");
166         map["arrLatitude"] = query.value("arr_latitude");
167         map["arrLongitude"] = query.value("arr_longitude");
168         map["startTime"] =
169             toLocalForDisplay(query.value("start_time")).toString("dd.MM.yyyy
170                 HH:mm");
171         map["endTime"] =
172             toLocalForDisplay(query.value("end_time")).toString("dd.MM.yyyy
173                 HH:mm");
174         map["startTimeUtcMs"] =
175             toUtcForCompare(query.value("start_time")).toMsecsSinceEpoch();
176         map["endTimeUtcMs"] =
177             toUtcForCompare(query.value("end_time")).toMsecsSinceEpoch();
178         map["status"] = statusText;
179         map["statusColor"] = statusColor;
180
181         // Te ID beda potrzebne przy edycji (zeby ustawic ComboBoxy)
182         map["planeId"] = query.value("plane_id");
183         map["depAirportId"] = query.value("departure_airport_id");
184         map["arrAirportId"] = query.value("arrival_airport_id");
185
186         list.append(map);
187     }
188     return list;
189 }

```

```

181
182 bool FlightManager::updateFlight(int id, int planeId, int
    depAirportId, int arrAirportId,
183                                     const QDateTime &startTime, const
                                         QDateTime &endTime) {
184     const QDateTime startUtc = toDbUtc(startTime);
185     const QDateTime endUtc = toDbUtc(endTime);
186
187     if (isPlaneInService(planeId)) {
188         qDebug() << "Nie mozna zaktualizowac lotu: samolot jest w
            serwisie.";
189         return false;
190     }
191
192     if (hasPlaneScheduleConflict(planeId, startUtc, endUtc, id)) {
193         qDebug() << "Nie mozna zaktualizowac lotu: samolot ma juz
            zajety termin.";
194         return false;
195     }
196
197     QSqlQuery query;
198     query.prepare("UPDATE flights SET plane_id = :pId,
        departure_airport_id = :depId, "
199                  "arrival_airport_id = :arrId, start_time = :start,
        end_time = :end "
200                  "WHERE id = :id");
201     query.bindValue(":pId", planeId);
202     query.bindValue(":depId", depAirportId);
203     query.bindValue(":arrId", arrAirportId);
204     query.bindValue(":start", startUtc);
205     query.bindValue(":end", endUtc);
206     query.bindValue(":id", id);
207
208     if (!query.exec()) {
209         qDebug() << "Blad SQL przy edycji lotu:" <<
            query.lastError().text();
210         return false;
211     }
212     return true;
213 }
214
215 bool FlightManager::deleteFlight(int id) {
216     QSqlQuery query;
217     query.prepare("DELETE FROM flights WHERE id = :id");
218     query.bindValue(":id", id);
219
220     if (!query.exec()) {
221         qDebug() << "Blad SQL przy usuwaniu lotu:" <<
            query.lastError().text();
222         return false;
223     }
224     qDebug() << "Pomyslnie usunieto lot o ID:" << id;

```

```
225     return true;
226 }
```

Listing 3: Operacje CRUD dla lotów (tabela `flights`) wraz z walidacją terminów

Struktura aplikacji

Aplikacja została zbudowana w typowym dla Qt podziale na warstwę logiki w C++ oraz warstwę interfejsu w QML. W praktyce oznacza to, że widoki (lista samolotów, lista lotów, statystyki itd.) są zdefiniowane w QML, natomiast pobieranie i modyfikacja danych odbywa się w C++ przez QSql. Ważnym elementem architektury jest sposób komunikacji C++-QML: w tym projekcie „klasy” nie pełnią roli tradycyjnych obiektów domenowych (z bogatym modelem typów i osobnymi klasami Plane/Flight/Airport), tylko są lekkimi **serwisami** (menedżerami) udostępniającymi zestaw metod do UI.

Warstwa C++: serwisy (menedżery)

W części C++ znajdują się klasy dziedziczące po QObject, których głównym zadaniem jest udostępnienie API do QML oraz komunikacja z bazą danych:

- **DatabaseManager** — inicjalizuje połączenie z PostgreSQL (Supabase) przez sterownik QPSQL.
- **PlaneManager** — operacje CRUD na samolotach oraz zapytania statystyczne (np. statystyki statusów i „ekstrema” floty).
- **AirportManager** — operacje CRUD na lotniskach oraz funkcje pomocnicze (np. parsowanie linku z Google Maps i podpowiedzi lotnisk).
- **FlightManager** — operacje CRUD na lotach wraz z walidacją terminów oraz pobieraniem danych z JOIN (samolot + lotniska).

Warto podkreślić, że nie są to „tradycyjne” klasy w znaczeniu obiektów przechowujących stan domeny w pamięci. Pełnią one rolę cienkiej warstwy pośredniej pomiędzy UI a bazą danych: wywołanie metody w QML uruchamia zapytanie SQL, a wynik jest konwertowany do typu zrozumiałego dla QML.

Udostępnienie serwisów do QML

Zamiast rejestrowania typów w silniku QML i budowy własnych modeli (np. QAbstractListModel), w projekcie zastosowano proste rozwiązanie: obiekty serwisów są wstrzykiwane do kontekstu QML jako contextProperty. Dzięki temu QML widzi je jako globalne obiekty (np. planeService) i może bezpośrednio wywoływać ich metody oznaczone jako Q_INVOKABLE.

```
1   QQuickStyle::setStyle("Material");
2   QGuiApplication app(argc, argv);
3   QGuiApplication::setWindowIcon(QIcon(u"qrc:/qt/qml/PlaneManager/assets/icon.png"));
4   QQmlApplicationEngine engine;
5   engine.setNetworkAccessManagerFactory(new
        OsmNetworkAccessManagerFactory());
6
7   loadDotEnv(QCoreApplication::applicationDirPath() + "../.env");
8
9   DatabaseManager dbManager;
10  dbManager.connectToSupabase(); // Test polaczenia z baza danych
11  engine.rootContext()->setContextProperty("myDb", &dbManager);
12
13  // Udostepniamy obiekt dla QML
```

```

14 PlaneManager planeManager;
15 engine.rootContext()->setContextProperty("planeService",
    &planeManager);
16
17 AirportManager airportManager;
18 engine.rootContext()->setContextProperty("airportService",
    &airportManager);
19
20 FlightManager flightManager;
21 engine.rootContext()->setContextProperty("flightService",
    &flightManager);
22
23 const QUrl url(u"qrc:/qt/qml/PlaneManager/qml/main.qml"_s);
24 QObject::connect(&engine, &QQmlApplicationEngine::objectCreated,
25     &app, [url](QObject *obj, const QUrl &objUrl) {
26         if (!obj && url == objUrl) QCoreApplication::exit(-1);
27     }, Qt::QueuedConnection);
28
29 engine.load(url);
30
31 return app.exec();
32 }

```

Listing 4: Udostępnienie obiektów serwisów w kontekście QML (fragment main.cpp)

Wymiana danych: QVariantList i QVariantMap

Kluczową cechą tej architektury jest dobór typów zwracanych przez metody serwisów. QML operuje na dynamicznych strukturach podobnych do JavaScript (tablice i obiekty), dlatego w C++ zwracane są typy generyczne:

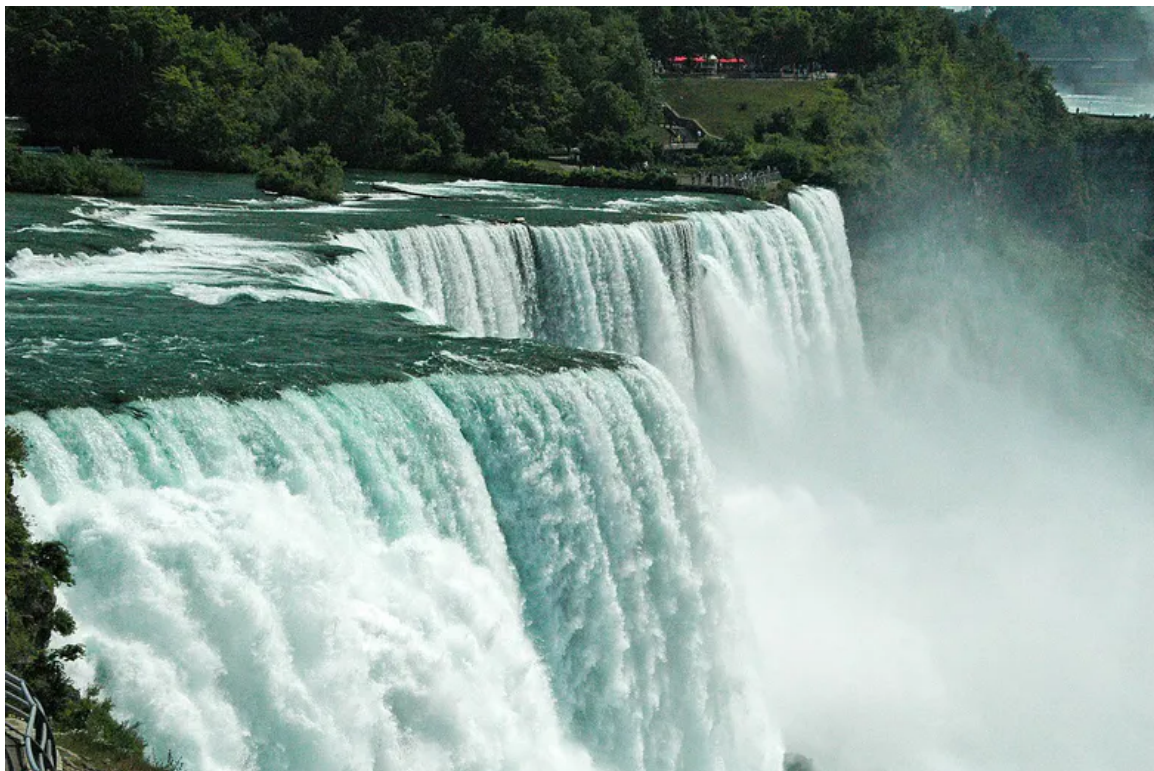
- **QVariantList** — lista rekordów (np. lista samolotów lub lotów),
- **QVariantMap** — pojedynczy rekord lub „obiekt” ze zbiorem pól (klucze–wartości),

W praktyce wygląda to następująco: C++ wykonuje zapytanie `QSqlQuery`, dla każdego wiersza tworzy mapę (np. {id, brand, model, status}), a następnie dodaje ją do listy. Po stronie QML rezultat jest przypisywany bezpośrednio jako model widoku listy, a w delegacie elementy są odczytywane przez klucze mapy (np. `modelData.brand`, `modelData.status`). To podejście znacząco upraszcza integrację z UI i redukuje ilość kodu „klejącego” (bez pisania osobnych klas modeli i rejestracji typów).

Minusem jest mniejsza kontrola typów w czasie kompilacji (łatwo o literówkę w nazwie klucza), jednak w projekcie o tej skali kompromis jest korzystny: logika pozostaje po stronie SQL/C++, a QML pełni rolę warstwy prezentacji.

Działanie aplikacji

W tym miejscu należy opisać końcowy rezultat oraz przedstawić screeny 2 pokazujące ważne momenty działania aplikacji.



Rysunek 2: Przykładowy zrzut ekranu

Podsumowanie

W tym miejscu należy podsumować projekt. Warto opisać co było najtrudniejszym elementem, jaka część była najciekawsza, a która pozwoliła się najwięcej nauczyć.

Jeżeli na jakimkolwiek etapie wykorzystywana była sztuczna inteligencja generatywna, to w tym miejscu należy również zamieścić link udostępniający pełną przeprowadzoną rozmowę(y). Link należy wygenerować, na samym końcu pisania raportu. Link powinien działać w momencie oceny pracy oraz w trakcie obrony projektu.